

# 实验 2: FPGA 布局算法实验报告

## 基于模拟退火的 HPWL 最小化布局

23336345 周海铭

2026 年 5 月 21 日

### 摘要

本实验基于模拟退火 (Simulated Annealing, SA) 算法实现了 FPGA 布局优化, 目标是最小化总线长 (Total HPWL)。算法采用双策略扰动机制 (交换 + 移动)、增量 HPWL 代价评估以及几何冷却策略。在 7 个不同规模的 benchmark 数据集上 (80 2000 个元件), 所有结果均通过布局合法性验证 (0 错误), 线长优化幅度达 40%~60%。

## 1 算法逻辑与实现思路

### 1.1 问题建模

FPGA 布局问题可形式化为: 给定  $W \times H$  的 FPGA 网格、一组固定 I/O 模块 (不可移动) 和一组可移动逻辑模块, 以及模块间的线网连接关系, 求一个从可移动模块到空闲 Block 坐标的映射, 使得目标函数最小:

$$\text{Minimize: } \Phi = \sum_{\text{all nets } n} \text{HPWL}(n) \quad (1)$$

其中 HPWL (Half-Perimeter Wire Length) 定义为线网连接的所有模块的包围盒半周长:

$$\text{HPWL}(n) = (\max_{i \in n} x_i - \min_{i \in n} x_i) + (\max_{i \in n} y_i - \min_{i \in n} y_i) \quad (2)$$

约束条件包括: (1) 固定模块坐标不可改变; (2) 每个 Block 最多容纳 1 个 Instance ( $\text{MAX\_BLOCK\_CAPACITY} = 1$ ); (3) Instance 坐标与 FPGA Block 状态必须双向联动。

## 1.2 算法选择：模拟退火

FPGA 布局是 NP-hard 组合优化问题，解空间随模块数呈阶乘增长。模拟退火算法通过概率接受劣解来跳出局部最优，是学术界标准 FPGA 布局工具 VPR 的核心布局引擎。相比解析法（二次布局 + 递归二分），SA 实现更简洁、对离散约束的处理更自然。

## 1.3 算法流程

**步骤 1: 数据收集:** 遍历 `glb_inst_map` 分离固定和可移动 Instance; 遍历 `glb_fpga` 收集空闲 Block 坐标。

**步骤 2: 随机初始布局:** 将  $N$  个可移动 Instance 随机分配到  $M$  个空闲 Block 中的  $N$  个 ( $M \geq N$ )，剩余  $M - N$  个 Block 保持空闲。

**步骤 3: SA 主循环:** 温度从  $T_0$  几何冷却至  $T_{\min}$ ，每个温度下执行  $N \times 10$  次扰动尝试。

## 1.4 约束处理

本问题的三项约束及其处理方式如下：

- (1) **固定模块不可移动:** `Instance::setPosition()` 内部检查 `isFixed()` 标志，对固定模块 (I/O Block) 调用时输出警告并忽略操作。SA 扰动时仅从 `movable_insts` 列表中选取，从根本上避免修改固定模块坐标。
- (2) **Block 容量限制** (`MAX_BLOCK_CAPACITY = 1`): 通过维护 `occupied_slots` (已占用位置列表) 和 `free_slots` (空闲位置列表) 两个数据结构来保证。每次扰动后执行 `glb_fpga.clearInst() + glb_fpga.addInst()` 原子式更新 Block 状态，天然保证每个 Block 至多容纳一个 Instance。
- (3) **Block-Instance 双向联动:** 交换/移动操作中同时更新 Instance 坐标 (`setPosition()`) 和 FPGA Block 的 `contain` 列表，确保 `reportValid()` 从 Instance→Block 和 Block→Instance 两个方向检查均一致。所有 7 个数据集均通过验证 (0 错误)。

## 1.5 扰动策略

本算法采用双策略混合扰动机制：

表 1: 扰动策略对比

| 策略               | 操作                                  | 适用场景                        |
|------------------|-------------------------------------|-----------------------------|
| 交换 (Swap)<br>70% | 随机选取两个已放置的 Instance, 交换其坐标          | 局部精细调整, 保持 Block 占用集合不变     |
| 移动 (Move)<br>30% | 随机选取一个已放置的 Instance, 将其移至随机空闲 Block | 全局探索, 改变 Block 占用集合, 跳出局部最优 |

每次扰动后执行 FPGA Block 双向联动: `glb_fpga.clearInst() → glb_fpga.addInst()`, 同时更新 Instance 的 `setPosition()`。这天然保证了 Block 容量约束。

## 1.6 增量代价计算

为避免每次迭代全量重算 HPWL, 仅评估包含被交换/移动 Instance 的 Net 集合:

$$\Delta\Phi = \sum_{n \in \mathcal{N}_a \cup \mathcal{N}_b} (\text{HPWL}_{\text{new}}(n) - \text{HPWL}_{\text{old}}(n)) \quad (3)$$

其中  $\mathcal{N}_a$ 、 $\mathcal{N}_b$  分别为 Instance  $a$  和  $b$  所连接的 Net 集合。复杂度从  $O(|\text{Nets}| \cdot d)$  降至  $O(k \cdot d)$ ,  $k \ll |\text{Nets}|$ 。

## 1.7 接受准则

$$P(\text{accept}) = \begin{cases} 1, & \Delta\Phi \leq 0 \\ \exp(-\frac{\Delta\Phi}{T}), & \Delta\Phi > 0 \end{cases} \quad (4)$$

## 1.8 SA 参数设计

表 2: SA 参数配置

| 参数              | 公式/值                                          | 说明                                |
|-----------------|-----------------------------------------------|-----------------------------------|
| 初始温度 $T_0$      | $\max(\Phi_{\text{initial}} \times 0.2, 100)$ | 自适应, 使初始接受概率 $\approx 80\%$       |
| 冷却系数 $\alpha$   | 0.95                                          | 几何冷却 $T_{k+1} = \alpha \cdot T_k$ |
| 终止温度 $T_{\min}$ | $T_0 \times 0.95^{150}$                       | 约 150 次降温后终止                      |
| 内循环次数           | $N \times 10$                                 | 与可移动模块数量成正比                       |
| 提前终止阈值          | $N \times 50$ 连续拒绝                            | 陷入局部最优时跳出                         |

## 1.9 创新点

- (1) **双策略混合扰动**: 交换 + 移动组合, 使 SA 既能精细调优又能全局探索, 特别适用于空闲 Block 多于可移动 Instance 的场景 ( $M > N$ )
- (2) **自适应初始温度**: 基于初始代价动态设定  $T_0$ , 避免手工调参
- (3) **增量 HPWL 评估**: 仅重算受影响 Net, 大幅提升大规模数据集的运行效率

## 2 运行结果

所有数据集均通过 `reportValid()` 验证 (0 错误), 布局完全合法。

表 3: 各数据集最终 HPWL 结果

| 数据集       | 元件数量  | FPGA 尺寸 | 固定模块 | 可移动模块 | 最终 HPWL       |
|-----------|-------|---------|------|-------|---------------|
| small.txt | ~80   | 10 × 10 | 25   | 51    | <b>455</b>    |
| med1.txt  | ~320  | 20 × 20 | 57   | 259   | <b>4268</b>   |
| med2.txt  | ~360  | 20 × 20 | 64   | 291   | <b>4889</b>   |
| lg1.txt   | ~720  | 30 × 30 | 89   | 627   | <b>17060</b>  |
| lg2.txt   | ~810  | 30 × 30 | 100  | 705   | <b>20154</b>  |
| xl.txt    | ~1280 | 40 × 40 | 121  | 1155  | <b>47569</b>  |
| huge.txt  | ~2000 | 50 × 50 | 153  | 1843  | <b>107947</b> |

## 3 实验总结

### 3.1 方法优点

- (1) **实现简洁**: 核心 SA 逻辑约 150 行 C++ 代码, 结构清晰易维护
- (2) **质量稳定**: 所有数据集均获得显著线长优化 (初始随机布局 → 最终收敛, 优化幅度 40%~60%)
- (3) **双策略扰动**: 交换 + 移动组合兼顾局部精细调优与全局探索
- (4) **增量代价计算**: 避免每次迭代全量重算 HPWL, 大数据集效率提升显著
- (5) **100% 合法性**: 所有输出通过 `reportValid()` 0 错误验证, Block-Instance 双向联动完全正确

### 3.2 方法缺点

- (1) **运行时间:** 大型数据集 (huge, ~2000 实例) 约需 2.9M 次迭代, 仍有优化空间
- (2) **参数敏感性:** SA 效果依赖初始温度和冷却策略, 不同数据集可能需要微调
- (3) **串行实现:** 未利用多核并行加速, 多起点 SA 可并行运行
- (4) **初始解随机:** 若采用贪心构造 (如按连接度排序优先放置), 可能加速收敛

### 3.3 下一步改进方向

- (1) **自适应温度调度:** 根据当前接受率动态调整冷却系数, 高温区快速降温、低温区精细搜索
- (2) **聚类引导布局:** 利用 Net 连接信息对高连接度 Instance 预分组, 缩小搜索空间
- (3) **多起点并行 SA:** 从多个随机初始解出发并行运行, 取最优结果
- (4) **与解析法结合:** 先用二次布局 (Quadratic Placement) 获得全局初始解, 再用 SA 精细优化
- (5) **HPWL 缓存优化:** 缓存每个 Net 的当前 HPWL 值, 交换时仅更新受影响的 Net 缓存, 进一步减少重复计算

## A 编译和运行命令

### 编译

```
g++ -std=c++17 -O2 -o main main.cpp Arch.cpp Global.cpp Object.cpp Solution.cpp
```

### 运行

```
./main benchmark/small.txt output/small_placement.txt  
./main benchmark/med1.txt output/med1_placement.txt  
./main benchmark/med2.txt output/med2_placement.txt  
./main benchmark/lg1.txt output/lg1_placement.txt  
./main benchmark/lg2.txt output/lg2_placement.txt  
./main benchmark/xl.txt output/xl_placement.txt  
./main benchmark/huge.txt output/huge_placement.txt
```